Análise Comparativa de Estruturas Disjoint Set Union: Da Abordagem Ingênua à Otimização de Tarjan

André X. Lazarini, João P. G. Ribeiro, Lucas F. Vilela e Marina F. S. Cabalzar

[1]Engenharia de software – Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brazil

{1147036, joao.ribeiro.1485762, lucas.vilela, marina.cabalzar}@sga.pucminas.br

Abstract. *This paper analyzes the impact of optimization heuristics on the Disjoint Set Union (DSU) data structure for solving the dynamic connectivity problem. Three versions were implemented in Java: Naive, Union by Rank, and Full Tarjan. Empirical experiments measured execution time and operation counts across worst-case, balanced, and random scenarios. The results validate the theoretical transition from $O(n)$ complexity to an almost constant amortized time $O(\alpha(n))$, highlighting the necessity of advanced data structures for large-scale algorithms.*

Resumo. *Este artigo analisa o impacto de heurísticas de otimização na estrutura de dados Disjoint Set Union (DSU) para a resolução do problema de conectividade dinâmica. Foram implementadas três variantes da estrutura em Java: Naive, Union by Rank e Full Tarjan. Experimentos empíricos mediram o tempo de execução e a contagem de operações em cenários de pior caso, árvores balanceadas e acessos aleatórios. Os resultados validam a transição teórica da complexidade $O(n)$ para um tempo amortizado quase constante $O(\alpha(n))$, evidenciando a necessidade de estruturas avançadas para algoritmos em larga escala.*

1. Introdução

O gerenciamento eficiente de subconjuntos disjuntos é fundamental para o desempenho de algoritmos de grafos e processamento de imagens. A estrutura Disjoint Set Union (DSU), também conhecida como Union-Find, busca solucionar o problema da conectividade dinâmica. A eficiência dessas operações impacta diretamente a viabilidade de algoritmos como o de Kruskal para Árvores Geradoras Mínimas.

Implementações básicas da DSU frequentemente sofrem com a degradação de desempenho em sequências longas de operações. Sem estratégias de balanceamento, a estrutura interna pode assumir formas lineares, elevando o custo da operação de busca ao nível $O(n)$. Este trabalho investiga como as técnicas introduzidas por Robert Tarjan, especificamente a união por rank e a compressão de caminho, reduzem a complexidade para um tempo amortizado quase constante.

Para comprovar tal ganho de eficiência, este experimento implementa a DSU em três níveis progressivos de otimização: a versão básica (Naive), a otimização por altura (Union by Rank) e a otimização completa (Full Tarjan). Através de uma suíte de benchmarks projetada para forçar cenários adversariais e aleatórios, o presente artigo realiza uma análise comparativa do tempo de execução e da quantidade de operações lógicas alocadas por cada abordagem.

2. Fundamentação Teórica

A análise de complexidade da DSU varia drasticamente conforme as heurísticas aplicadas. Na implementação *Naive*, a ausência de critérios de união resulta em árvores desbalanceadas com altura proporcional ao número de elementos n. Consequentemente, o limite superior de tempo para a operação *Find* é $O(n)$, tornando a estrutura ineficiente para grandes volumes de dados.

A introdução da união por rank (*Union by Rank*) limita a altura da árvore ao garantir que o conjunto menor seja sempre anexado à raiz do conjunto maior. Essa restrição estrutural assegura que a altura máxima não ultrapasse $\log n$, reduzindo a complexidade para $O(\log n)$. Entretanto, o desempenho ideal é atingido apenas com a adição da compressão de caminho (*Path Compression*).

A técnica de Tarjan utiliza a compressão de caminho para achatar a estrutura da árvore durante cada execução da operação *Find*. A combinação de ambas as heurísticas resulta em uma complexidade amortizada de $O(\alpha(n))$, onde α é a inversa da função de Ackermann. Na prática, essa função cresce de forma tão lenta que seu valor permanece inferior a 5 para qualquer entrada fisicamente computável.

3. Metodologia

3.1. Implementação e Tecnologias

O projeto foi desenvolvido em Java, com a arquitetura centralizada na interface `DisjointSet`, que estabelece o contrato das operações fundamentais. O uso de métodos *default* para a instrumentação (contagem de operações) habilitou o uso de polimorfismo na classe de testes (`Benchmark`), garantindo que o algoritmo avaliador fosse agnóstico em relação à implementação interna da estrutura, facilitando a extensibilidade e blindando os testes contra vieses. O ambiente de execução consistiu em um processador Intel(R) Core(TM) i7-10700F CPU @ 2.90GHz, com 24 GB de RAM, operando sobre o sistema Windows 11. Os testes foram compilados e executados utilizando a JVM OpenJDK 24.0.1 (Temurin).

3.2. Cenários de Teste

A avaliação empírica utilizou a classe `Benchmark` para simular três cenários distintos:

- **Pior Caso (Cadeia Linear):** Força a criação de uma lista encadeada através de uniões sequenciais de i com $i + 1$, maximizando a profundidade da busca.
- **Árvore Balanceada:** Constrói uma estrutura perfeitamente equilibrada para observar o comportamento logarítmico.
- **Aleatório:** Simula o uso típico da estrutura com uniões e buscas baseadas em sementes aleatórias fixas para reprodutibilidade.

Cada teste foi precedido por uma fase de aquecimento da JVM para estabilizar o compilador JIT e executado em 3 rodadas (*trials*) para mitigar ruídos térmicos ou do sistema operacional.

4. Discussão de Resultados

4.1. Desempenho no Pior Caso (Adversarial)

O cenário adversarial demonstra o impacto severo da falta de balanceamento. O tempo de execução da versão Naive apresenta um crescimento linear acentuado, alcançando a

marca de quase 100 ms para entradas maiores. Em contrapartida, as implementações Union by Rank e Full Tarjan mantêm o tempo de processamento praticamente constante e próximo a 0 ms, evidenciando a eficácia imediata das heurísticas.

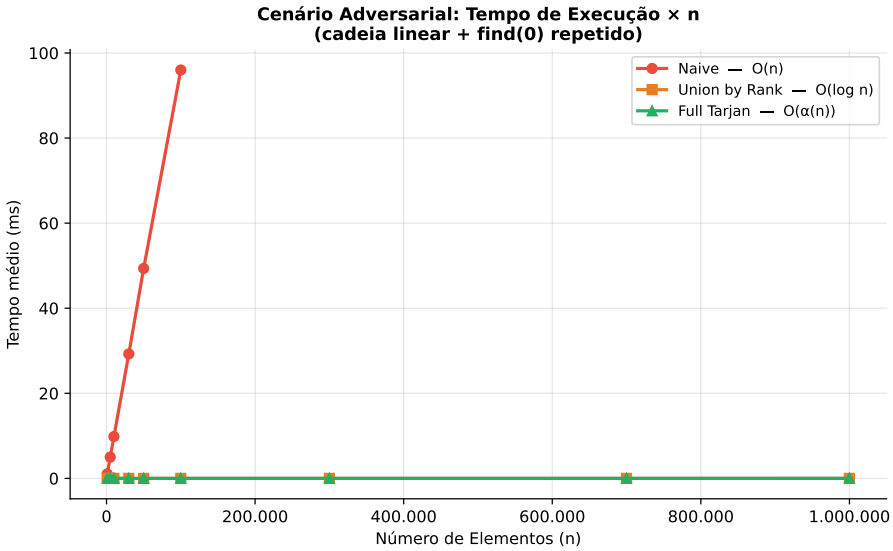

Figura 1. Tempo de execução no cenário adversarial (cadeia linear).

A análise das operações em memória corrobora o impacto temporal. A contagem de operações por busca evidencia a classe de complexidade $O(n)$ da versão Naive. As abordagens otimizadas mitigam a formação das cadeias longas, achatando a curva de forma estrutural e mantendo os acessos mínimos.

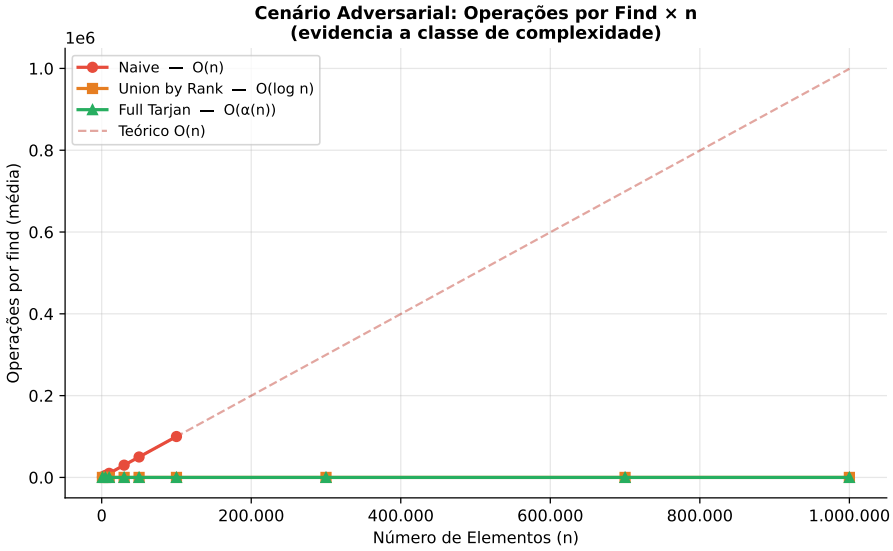

Figura 2. Média de operações por busca no cenário adversarial.

4.2. Desempenho em Árvores Balanceadas

No cenário de árvore construída de forma balanceada, a versão Union by Rank exibe um crescimento visivelmente logarítmico, operando paralela à sua complexidade teórica

de $O(\log n)$. A adoção da compressão de caminhos na implementação Full Tarjan elimina essa dependência de profundidade logarítmica, sustentando a contagem média de operações em um platô inferior, validando a complexidade $O(\alpha(n))$.

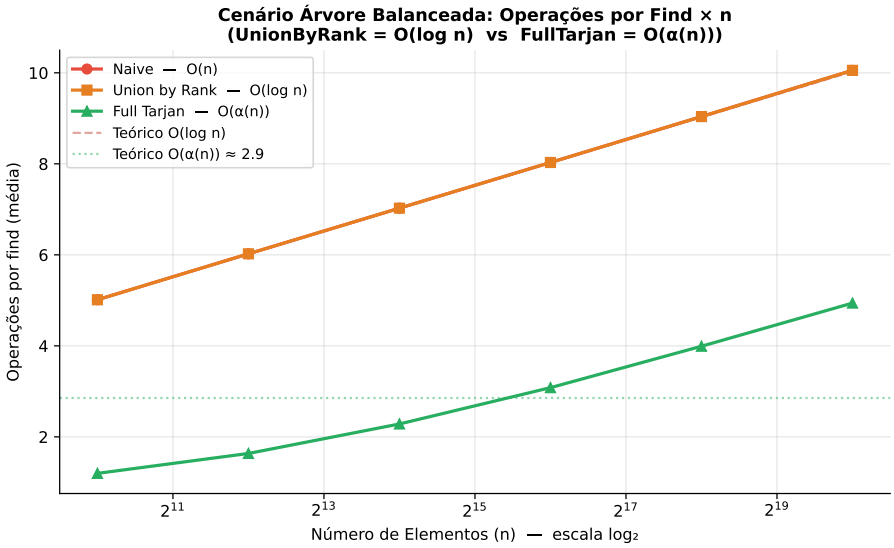

Figura 3. Crescimento de operações por busca no cenário balanceado.

4.3. Desempenho no Cenário Aleatório

A simulação de requisições aleatórias reflete cenários de aplicação convencionais de algoritmos de conectividade dinâmica. A versão Naive sofre rápida degradação inicial de tempo e torna-se inviável em larga escala. As versões otimizadas absorvem bem a carga de até 1.000.000 de elementos, contudo, a implementação Full Tarjan consegue entregar tempos médios de resposta ligeiramente inferiores aos da Union by Rank devido ao processo contínuo de encurtamento das árvores em tempo de execução.

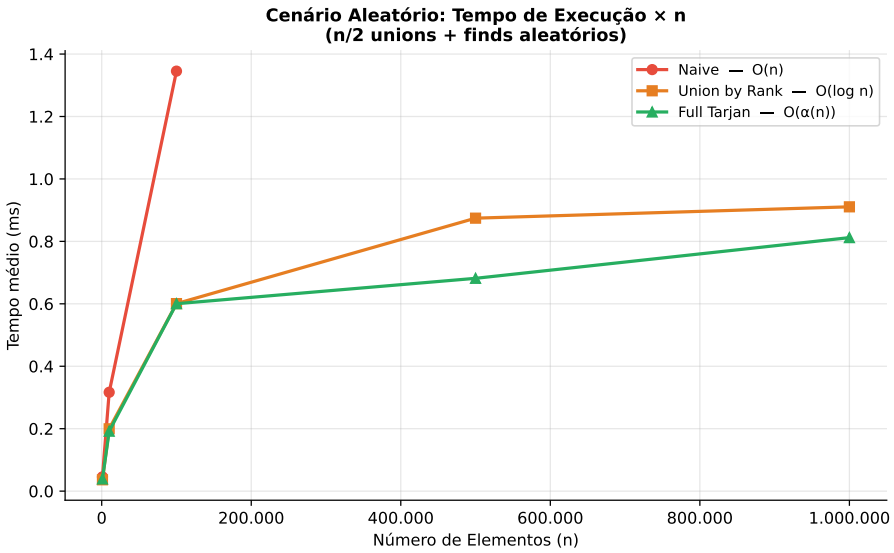

Figura 4. Tempo médio de resposta em operações aleatórias.

5. Conclusão

Os experimentos realizados confirmam que a escolha adequada de estruturas de dados é determinante para a escalabilidade de sistemas de software. A implementação básica da DSU mostrou-se inviável para grandes conjuntos de dados, apresentando degradação severa em cenários lineares. Em contrapartida, as heurísticas de Tarjan demonstraram eficiência excepcional, mantendo o desempenho quase constante mesmo com milhões de elementos. A transição da complexidade de $O(n)$ para $O(\alpha(n))$ representa não apenas um ganho quantitativo, mas uma mudança de viabilidade algorítmica essencial para a engenharia de software moderna.